WEEK 9 -- SQL: Aggregation and GROUP BY

Setup: *Use the robotics database from 01_setup.sql. Today is 2026-10-01; September means [2026-09-01, 2026-10-01). All mission dates use started_at. Completed cost includes only completed missions; costs are PLN and timestamps UTC.*

1. In-Class Exercises

1.1. IC9.1 -- Group operator identity correctly

For September missions, report operator_id, operator name, mission_count, and completed_cost, keeping guest missions in one NULL-operator group. Explain why grouping only by missions.operator_id does not permit selecting the joined operator's name, and why grouping only by name could merge operators. Write the corrected query, use zero for groups without completed missions, and sort operator_id NULLS LAST.

1.2. IC9.2 -- Six views of mission activity

For September missions, return all mission rows, registered-operator mission rows, distinct registered operators, guest mission rows, known-status rows, and distinct operators with completed missions. All six counts must be zero on an empty input. Explain why subtracting guest missions from all missions does not count distinct operators.

1.3. IC9.3 -- Qualifying daily operating costs

Group completed September missions by their UTC start date. Keep only dates with at least three completed missions and at least 500 PLN in completed mission cost. Return mission_day, completed_missions, completed_cost, and average_completed_mission rounded to two decimals, ordered by date. Apply thresholds to the groups rather than individual mission costs.

1.4. IC9.4 -- Country and mission-status subtotals

Use ROLLUP to summarize September missions by country and status, by country alone, and overall. Guests and unknown operator countries share a real NULL-country group. Show both GROUPING flags, mission_count, and completed_cost, using zero when no completed missions qualify. Put real NULL keys first, each country subtotal after its detail rows, and the grand total last.

1.5. IC9.5 -- Every operator's mission profile

Return every operator's id and name with September total_missions, completed_missions, queued_missions, aborted_missions, unknown_status_missions, and completed_cost. Use conditional aggregation after a LEFT JOIN, preserving operators without missions with zeros. Count SQL NULL status only for actual missions; an unmatched outer-join row must not count as a mission. Sort by operator id.

1.6. IC9.6 -- Highest operating cost per country

Calculate every operator's September completed mission cost, using zero for operators without completed missions. Return every operator tied for the highest total in their country, with NULL countries treated as one group. Guest missions belong to no registered operator. Show operator_id, name, country, and completed_cost; use NOT EXISTS over grouped totals and sort country NULLS FIRST, then id.

1.7. IC9.7 -- Mission-cost medians with repeated values

For every operator, show id, name, September completed-mission count, and median cost rounded to two decimals. Repeated costs remain separate observations; average the two middle observations for even counts. Operators without completed missions have count zero and NULL median. Use grouped frequencies and a self join to find occupied rank intervals, without percentile functions or windows. Sort by operator id.

1.8. IC9.8 -- Month-to-date mission costs

Return all 30 September dates with daily completed_cost and month_to_date_cost through that date inclusive. Keep dates without completed missions using daily cost zero; the cumulative total must carry forward. Build a calendar and daily grouped totals, then use a correlated aggregate for the running sum, without window functions. Order by mission_day.

1.9. IC9.9 -- Monthly robotics costs with GROUPING SETS

For completed missions in [2026-04-01, 2026-10-01), produce month-country detail, monthly subtotals, and a grand total using GROUPING SETS. Return mission_month, country, both GROUPING flags, completed_missions, and completed_cost. Do not add country-only subtotals or empty months. Combine guests with unknown countries; order months ascending, detail before subtotal, real NULL countries first, and grand total last.

1.10. IC9.10 -- A seven-day robotics dashboard

For September 24-30, return each date's completed_missions, completed_cost, active_operators, new_active_operators, returning_active_operators, and top_country. New means first-ever completed mission on that date; returning means an earlier first completion. Count registered operators distinctly; guests contribute missions and costs. Keep empty dates with zeros and NULL top_country. Rank countries by daily completed cost descending, then country NULLS LAST; order dates ascending.

2. Homework Exercises

2.1. HW9.1 -- Missing sensor values in aggregates

Using sensor_readings, return row_count, nonnull_count, total_value, nonnull_average, minimum_value, maximum_value, and null_as_zero_average. Round both averages to two decimals; replace NULL with zero only inside the final average. Explain why COUNT(*) differs from COUNT(value), and why treating a missing sensor observation as zero changes the average's denominator. Return one summary row.

2.2. HW9.2 -- Mission-status percentages by country

Group September missions by operator country and status, combining guests with unknown countries. Return country, status, mission_count, and country_mission_pct rounded to two decimals, using all missions in that country as the denominator. Include unknown statuses as real groups and compare NULL countries safely. Sort country NULLS FIRST, then status NULLS FIRST; do not invent groups without missions.

2.3. HW9.3 -- First and last missions

For every operator, report id, name, total mission count before 2026-10-01, and the id, start timestamp, and cost of their first and last mission, across all statuses. Resolve timestamp ties by smaller id for first and larger id for last. Keep operators without missions with count zero and NULL details. Use grouped counts and lateral lookups; sort by operator id.

2.4. HW9.4 -- Robot-model deployment report

For every robot model, show id, name, September mission_count, distinct_registered_operators across all statuses, distinct_active_operators using completed missions only, completed_units, and completed_line_cost. Count missions containing the model, not robot units; calculate completed costs from historical unit_cost times robot_count. Guests contribute missions, units, and costs but no operator counts. Keep unused models with zeros and order by model id.

2.5. HW9.5 -- Mission costs in five buckets

Classify all September mission costs into [0,100), [100,250), [250,500), [500,1000), and [1000,+infinity), numbered 1-5. Lower bounds are inclusive and finite upper bounds exclusive. Return bucket_no, cost_bucket, mission_count, completed_missions, and completed_cost. Preserve all five buckets with zeros when empty. Order by bucket number; an exact boundary belongs to the higher bucket.

2.6. HW9.6 -- Operator-specific cost anomalies

For operators with at least five completed missions before 2026-10-01, calculate mean cost and population standard deviation from all those missions. Return completed missions strictly above their operator's mean plus twice that deviation. Show operator_id, mission_id, cost, mean_cost, population_sd, and z_score. Round statistics to two decimals and z_score to three, but filter full-precision values. Order operator, cost descending, mission id.

2.7. HW9.7 -- Pivot robot monitoring metrics

For September 24-30, pivot robot_metrics into sensor_reads, faults, and missions columns. Keep every date, leaving missing rows and NULL observations as SQL NULL. Also calculate missions_per_sensor_read_pct as 100 times missions divided by sensor_reads, rounded to two decimals, returning NULL when sensor_reads is zero or missing. Use conditional aggregates and order by metric_day.

2.8. HW9.8 -- Most-used robot models per operator

Aggregate registered operators' completed September deployments by operator and model. Return up to three models per operator, ranked by completed_units descending, completed_line_cost descending, then model id ascending. Return operator_id, operator_name, robot_model_id, robot_model_name and both measures, using historical deployment costs. Resolve all ties so no operator exceeds three rows. Use a correlated count of better groups rather than windows.

2.9. HW9.9 -- Twenty-four hours of mission activity

Report every UTC hour of 2026-09-30 as a half-open one-hour bucket. Return hour_start, total_missions, completed_missions, guest_missions, and completed_cost; guests have NULL operator_id. Keep all 24 hours with zeros when empty, ensuring an unmatched outer-join row counts as neither a mission nor a guest. Order by hour_start.

2.10. HW9.10 -- Operator cohort retention

Group operators registered before 2026-10-01 by registration calendar month, retaining soft-deleted operators in cohort_size. Return offsets 0-5 only when activity_month starts before the cutoff. Active means at least one completed mission in that month, at or after the actual registration timestamp; count each operator once. Return cohort_month, month_offset, activity_month, cohort_size, active_operators, and retention_pct rounded to two decimals. Preserve observed zero-activity cells; sort cohort and offset.